

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ - ĐHQGHN

VIỆN TRÍ TUỆ NHÂN TẠO

BÁO CÁO KẾT THÚC MÔN
CƠ SỞ TRÍ TUỆ NHÂN TẠO

Huấn luyện mô hình cờ vua

Thành viên nhóm: Trương Ái Linh
Nguyễn Quốc Khánh
Cao Huy Hòa
Nguyễn Thị Hiền Trang

Tháng 6, 2026

Mục lục

1	Đặt vấn đề và lý do chọn đề tài	2
2	Thuật toán	2
2.1	Mã hóa bàn cờ và xử lý dữ liệu	2
2.2	Khái quát thuật toán	3
2.3	Giai đoạn 1: Mô hình mạng nơ-ron học có giám sát	3
2.3.1	Kiến trúc mô hình ChessNet	3
2.3.2	Hàm mất mát	5
2.3.3	Các kỹ thuật tối ưu trong huấn luyện	7
2.3.4	Kết quả sau huấn luyện	7
2.4	Giai đoạn 2: MCTS và học tăng cường	8
2.4.1	Monte Carlo Tree Search (MCTS)	8
2.4.2	Batched MCTS — tối ưu hóa phần cứng	10
2.4.3	Bước 1 — Mô hình tự chơi với chính mình	10
2.4.4	Bước 2 — Huấn luyện học tăng cường	11
2.4.5	Bước 3 — So sánh và cập nhật best model	11
2.5	Các cơ chế tối ưu tốc độ và phần cứng	12

1. Đặt vấn đề và lý do chọn đề tài

Cờ vua là một trong những trò chơi trí tuệ có lịch sử lâu đời của nhân loại, đồng thời là một bài toán kinh điển trong lĩnh vực trí tuệ nhân tạo. Không gian trạng thái của cờ vua vô cùng rộng lớn với khoảng 10^{120} nước đi có thể xảy ra — lớn hơn số nguyên tử trong vũ trụ quan sát được — khiến cho các phương pháp tìm kiếm vét cạn truyền thống hoàn toàn bất khả thi.

Sự ra đời của AlphaGo (2016) và đặc biệt là AlphaZero (2017) của DeepMind đã đánh dấu một bước ngoặt lịch sử: một hệ thống AI có thể tự học chơi cờ vua từ đầu, không cần bất kỳ kiến thức con người nào ngoài luật chơi, và đạt trình độ siêu đẳng chỉ trong vài giờ. Kiến trúc này kết hợp mạng nơ-ron sâu với thuật toán tìm kiếm Monte Carlo Tree Search (MCTS), tạo ra một vòng phản hồi tự cải thiện cực kỳ mạnh mẽ.

Nhóm chúng em chọn đề tài này nhằm tìm hiểu và tái hiện pipeline học máy theo tinh thần AlphaZero trên quy mô vừa phải, bao gồm: xây dựng mạng nơ-ron tùy chỉnh, huấn luyện có giám sát trên dữ liệu ván đấu thực tế của các kỳ thủ chuyên nghiệp, rồi tinh chỉnh thêm bằng học tăng cường thông qua tự chơi. Đây là cơ hội áp dụng trực tiếp các kiến thức từ môn học — từ biểu diễn tri thức, tìm kiếm heuristic, đến học máy — vào một bài toán cụ thể và có thể đánh giá định lượng được.

2. Thuật toán

2.1. Mã hóa bàn cờ và xử lý dữ liệu

Để đưa trạng thái bàn cờ vào mạng nơ-ron, cần biểu diễn nó dưới dạng một tensor số có cấu trúc rõ ràng. Nhóm sử dụng định dạng $15 \times 8 \times 8$, tức là 15 lớp “ảnh” kích thước 8×8 xếp chồng nhau, mỗi lớp mô tả một khía cạnh của trạng thái bàn cờ.

Cấu trúc 15 kênh đầu vào:

- **Kênh 1–6:** Vị trí các quân Trắng (Tốt, Mã, Tượng, Xe, Hậu, Vua). Ô có quân = 1, không có = 0.
- **Kênh 7–12:** Tương tự cho các quân Đen.
- **Kênh 13:** Tất cả các ô = 1 (flag báo hiệu đây là góc nhìn của người đang đến lượt đi).
- **Kênh 14–15:** Đánh dấu ô xuất phát và ô đích của nước đi liền trước, giúp mạng nắm được lịch sử ngắn hạn.

Canonical Flip. Bàn cờ luôn được lật sao cho người đang đến lượt đi luôn ở phía

dưới. Khi đến lượt Đen, toàn bộ bàn cờ được lật dọc và màu quân được hoán đổi. Nhờ đó mạng không cần học hai chiến lược riêng biệt cho Trắng và Đen — giảm một nửa chiều phức tạp của bài toán.

Lọc và xử lý dữ liệu PGN:

- Chỉ giữ các ván cờ có ít nhất 40 lượt đi (≥ 20 nước mỗi bên), loại bỏ ván quá ngắn hoặc lỗi.
- Bỏ qua 8–10 lượt đầu (khai cuộc) và 8–10 lượt cuối (tàn cuộc hỗn loạn) để tập trung vào phần trung cuộc có chiến thuật rõ ràng nhất.
- Lấy 1 vị trí mỗi 2 lượt với offset ngẫu nhiên, nghĩa là trong 1 ván chỉ lấy toàn nước đi của trắng hoặc của đen, đảm bảo tỉ lệ cân bằng giữa góc nhìn Trắng và Đen sau Canonical Flip.

2.2. Khái quát thuật toán

Hệ thống được xây dựng theo hai giai đoạn liên tiếp, phản ánh cách học của con người khi học cờ: trước tiên học từ sách vở và ván đấu mẫu, sau đó tự luyện tập để hoàn thiện.

- **Giai đoạn 1 — Học có giám sát (Supervised Learning):** huấn luyện mạng nơ-ron trên dữ liệu thực tế của 250.000 ván cờ (xấp xỉ 9 triệu vị trí bàn cờ) từ các kỳ thủ hàng đầu thế giới (Lichess Elite Database). Mục tiêu là khởi tạo một mô hình có hiểu biết cờ vua cơ bản tốt trước khi bước vào giai đoạn tự chơi.
- **Giai đoạn 2 — Học tăng cường (Reinforcement Learning):** áp dụng thuật toán MCTS để mô hình tự chơi với chính mình, sinh ra dữ liệu mới, và liên tục cải thiện bản thân qua vòng lặp phản hồi.

2.3. Giai đoạn 1: Mô hình mạng nơ-ron học có giám sát

2.3.1. Kiến trúc mô hình ChessNet

Mạng nơ-ron được thiết kế theo kiến trúc *song đầu* (dual-head), gồm ba phần chính: lớp đầu vào, backbone trích xuất đặc trưng, và hai nhánh đầu ra độc lập.

Lớp đầu vào (Input Conv). Một lớp tích chập 3×3 nhận tensor $15 \times 8 \times 8$ và chiếu lên không gian 64 kênh đặc trưng, tiếp theo là Batch Normalization và hàm kích hoạt ReLU. Đây là bước “dịch” bàn cờ sang ngôn ngữ nội tại của mạng.

Backbone — ResBlock với Squeeze-Excite gate. Backbone gồm 6 khối ResBlock xen kẽ với 2 lớp BoardAttention (đặt sau khối 3 và khối 6). Mỗi ResBlock có cấu trúc:

$$\text{output} = \text{ReLU}(\text{Conv}_2(\text{Conv}_1(x)) \times \mathbf{w}_{\text{SE}} + x) \quad (1)$$

Trong đó x là input, $\text{Conv}_1, \text{Conv}_2$ là hai lớp tích chập 3×3 liên tiếp có Batch Normalization, và $\mathbf{w}_{\text{SE}} \in (0, 1)^{64}$ là vector trọng số từ cơ chế Squeeze-Excite. *Skip connection* ($+x$) đảm bảo thông tin ban đầu không bị mất khi đi qua nhiều tầng — đây là ý tưởng cốt lõi của kiến trúc ResNet.

Cơ chế Squeeze-Excite (SE gate). SE gate là một cơ chế chú ý theo kênh, giúp mạng tự động điều chỉnh tầm quan trọng của từng trong 64 kênh đặc trưng dựa trên nội dung toàn cục của bàn cờ. Cơ chế hoạt động qua ba bước:

1. **Squeeze:** Nén toàn bộ feature map $64 \times 8 \times 8$ thành vector $\mathbf{z} \in \mathbb{R}^{64}$ bằng Global Average Pooling:

$$z_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W x_c(i, j), \quad c = 1, \dots, 64 \quad (2)$$

2. **Excite:** Đưa $\mathbf{z}$ qua hai lớp FC nhỏ ($64 \rightarrow 16 \rightarrow 64$) với ReLU và Sigmoid để tạo vector trọng số kênh $\mathbf{w} \in (0, 1)^{64}$:

$$\mathbf{w} = \sigma(W_2 \text{ReLU}(W_1 \mathbf{z})) \quad (3)$$

3. **Scale:** Nhân từng kênh của feature map với trọng số tương ứng:

$$\hat{x}_c = x_c \times w_c \quad (4)$$

Kênh nào $w_c \approx 1$ được giữ nguyên, kênh nào $w_c \approx 0$ bị tắt gần hết. Ví dụ, trong tàn cuộc không còn Mã, kênh đặc trưng cho Mã sẽ tự động bị giảm trọng số. Toàn bộ cơ chế này chỉ tốn $64 \times 16 + 16 \times 64 = 2048$ tham số nhưng mang lại cải thiện đáng kể về chất lượng đặc trưng.

BoardAttention — chú ý toàn cục. Tích chập thông thường chỉ nhìn được vùng lân cận của mỗi ô (receptive field cố định), không nắm bắt được mối quan hệ giữa các quân cờ ở xa nhau — ví dụ Hậu ở góc A8 kiểm soát đường chéo xuống B1. BoardAttention giải quyết vấn đề này bằng cách:

- Trải phẳng bàn cờ 8×8 thành chuỗi 64 phần tử (mỗi phần tử là vector 64 chiều đặc trưng của một ô).
- Áp dụng Multi-Head Self-Attention với 4 đầu, cho phép mỗi ô chú ý tới tất cả 63

ô còn lại cùng một lúc — tương tự cơ chế Transformer trong NLP.

- Sau attention: Layer Normalization, FFN (Feed-Forward Network), Layer Normalization lần hai để ổn định gradient.

Policy Head — dự đoán nước đi. Policy Head dự đoán phân phối xác suất trên không gian $64 \times 64 = 4096$ nước đi có thể (tích Descartes ô xuất phát $\times$ ô đích). Luồng xử lý:

$$\underbrace{64 \times 8 \times 8}_{\text{feature map}} \xrightarrow{\text{Conv}_{1 \times 1}} \underbrace{32 \times 8 \times 8} \xrightarrow{\text{BN+ReLU+Flatten}} 2048 \xrightarrow{\text{FC}} \underbrace{4096 \text{ logits}}_{\text{Policy output}} \quad (5)$$

Trước khi đưa qua Softmax, các nước đi bất hợp lệ bị đặt về $-\infty$ thông qua *Legal Mask*, đảm bảo mạng chỉ gán xác suất cho các nước đi đúng luật:

$$\tilde{l}_i = l_i + m_i, \quad m_i = \begin{cases} 0 & \text{nếu nước đi } i \text{ hợp lệ} \\ -\infty & \text{nếu bất hợp lệ} \end{cases} \quad (6)$$

Value Head — đánh giá thế cờ (WDL). Value Head dự đoán kết quả dưới dạng phân phối Win-Draw-Loss (WDL) thay vì một giá trị vô hướng đơn lẻ như trong AlphaZero gốc. Luồng xử lý:

$$\underbrace{64 \times 8 \times 8}_{\text{feature map}} \xrightarrow{\text{Conv}_{1 \times 1}} \underbrace{8 \times 8 \times 8} \xrightarrow{\text{Flatten}} 512 \xrightarrow{\text{FC}} 64 \xrightarrow{\text{FC}} \underbrace{[P(\text{Win}), P(\text{Draw}), P(\text{Loss})]}_{\text{WDL output}} \quad (7)$$

Khi cần một giá trị vô hướng để so sánh các nước đi trong MCTS:

$$v_{\text{scalar}} = P(\text{Win}) - P(\text{Loss}) \in [-1, +1] \quad (8)$$

Cách tiếp cận WDL cho phép mô hình phân biệt rõ ràng ba tình huống mà một con số trong $[-1, 1]$ khó biểu diễn chính xác — đặc biệt quan trọng để phân biệt “đang dẫn nhưng có thể hòa” với “thắng chắc”.

2.3.2. Hàm mất mát

Mô hình được tối ưu hóa đồng thời theo hai mục tiêu thông qua hàm mất mát tổng hợp:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{policy}} + \lambda \cdot \mathcal{L}_{\text{value}} \quad (9)$$

Với $\lambda = 1,0$ (VALUE_WEIGHT), hai đầu ra được coi trọng ngang nhau.

Policy Loss — Cross-Entropy với Label Smoothing. Để tránh mô hình quá tự tin vào một nước đi duy nhất (overfitting), nhãn cứng (hard label) được làm mềm với hệ số ε :

$$y_{\text{smooth}}(i) = \begin{cases} 1 - \varepsilon & \text{nếu } i \text{ là nước đúng} \\ \frac{\varepsilon}{K - 1} & \text{các nước còn lại} \end{cases} \quad (10)$$

Hàm mất mát Policy là cross-entropy giữa nhãn đã làm mềm và phân phối dự đoán sau masking:

$$\mathcal{L}_{\text{policy}} = - \sum_{i=1}^K y_{\text{smooth}}(i) \log(\text{softmax}(\tilde{l}_i)) \quad (11)$$

Với $K = 4096$ và $\tilde{l}_i$ là logit sau khi áp Legal Mask.

Value Loss — Cross-Entropy WDL. Value Head được huấn luyện bằng Cross-Entropy tiêu chuẩn, nhãn là class kết quả thực tế từ góc nhìn người đang đến lượt:

$$\mathcal{L}_{\text{value}} = -\log(p[c]), \quad c \in \{0(\text{Win}), 1(\text{Draw}), 2(\text{Loss})\} \quad (12)$$

2.3.3. Các kỹ thuật tối ưu trong huấn luyện

Bảng 1: Các kỹ thuật tối ưu hóa áp dụng trong quá trình huấn luyện

Kỹ thuật	Mô tả và tác dụng
AdamW	Kết hợp Adam với L2 Regularization (weight decay = 10^{-4}). Ngăn trọng số phình to và giảm nguy cơ overfitting, phù hợp hơn Adam thuần cho mô hình có Attention.
Mixed Precision (AMP)	Dùng float16 cho các phép tính nặng thay vì float32, tiết kiệm VRAM và tăng tốc huấn luyện gần 2×. Đặc biệt cần thiết khi train trên GPU T4/A100 có VRAM giới hạn.
GradScaler	Tự động khuếch đại gradient để tránh underflow khi dùng float16. Sau mỗi bước, gradient được unscale về float32 trước khi clipping và cập nhật.
Gradient Clipping	Giới hạn L2-norm của vector gradient không vượt quá 1,0: $\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{1}{\ \mathbf{g}\ }\right)$. Ngăn exploding gradient gây mất ổn định huấn luyện.
Warmup Cosine LR	+ Lch trình 2 giai đoạn: (1) tăng tuyến tính trong 1000 bước đầu tránh gradient lớn khi mạng chưa ổn định; (2) giảm dần theo Cosine để hội tụ mượt mà.
Label Smoothing	Lm mềm nhãn one-hot với ε nhỏ, phân bổ một phần xác suất nhỏ cho các nước đi khác. Giảm overfitting vào phong cách chơi của từng kỳ thủ cụ thể.
Chunk DataLoader	Chia 9 triệu vị trí thành các chunk ~ 30.000 vị trí, nạp tuần tự vào RAM. Tránh tràn bộ nhớ trên môi trường Colab/Kaggle.

Lch trình learning rate theo hai giai đoạn được mô tả qua các công thức:

$$\text{lr}(t) = \begin{cases} \text{LR}_{\max} \cdot \frac{t}{t_{\text{warmup}}} & t < t_{\text{warmup}} \\ \text{LR}_{\min} + \frac{1}{2}(\text{LR}_{\max} - \text{LR}_{\min}) \left(1 + \cos\left(\pi \cdot \frac{t - t_{\text{warmup}}}{T - t_{\text{warmup}}}\right)\right) & t \geq t_{\text{warmup}} \end{cases} \quad (13)$$

Trong đó T là tổng số bước huấn luyện, $t_{\text{warmup}} = 1000$, $\text{LR}_{\max} = 10^{-3}$, $\text{LR}_{\min} = 10^{-5}$.

2.3.4. Kết quả sau huấn luyện

Sau 7 epoch huấn luyện trên tập dữ liệu 250.000 ván cờ, mô hình đạt được:

- **Top-1 Accuracy: 46%** — trong 46% trường hợp, nước đi mô hình dự đoán khớp chính xác với nước đi của kỳ thủ chuyên nghiệp.
- **Top-5 Accuracy: 86%** — nước đi của kỳ thủ nằm trong 5 nước được mô hình xếp hạng cao nhất.
- **Đấu với Stockfish ELO 1400** qua 50 ván: winrate đạt **54%**.

Top-5 accuracy 86% cho thấy mô hình đã nắm bắt được tư duy chiến thuật ở mức tốt: phần lớn các tình huống, mô hình hiểu được các nước đi “có lý” dù chưa luôn chọn đúng nước tối ưu nhất. Đây là nền tảng tốt để bước vào giai đoạn học tăng cường.

2.4. Giai đoạn 2: MCTS và học tăng cường

Sau khi có mô hình nền từ giai đoạn 1, nhóm áp dụng học tăng cường theo tinh thần AlphaZero: cho mô hình tự chơi với chính mình, sử dụng MCTS để nâng cao chất lượng nước đi, và học từ kết quả ván đấu thực tế thay vì từ dữ liệu kỳ thủ người.

2.4.1. Monte Carlo Tree Search (MCTS)

MCTS là thuật toán lập kế hoạch cho phép mô hình “suy nghĩ trước” bằng cách mô phỏng hàng trăm ván cờ giả định trong đầu trước khi quyết định nước đi. Thay vì tin tưởng vào output của policy head, MCTS xây dựng một cây tìm kiếm và tinh chỉnh lại phân phối dựa trên kết quả mô phỏng.

Mỗi simulation gồm 4 bước lặp:

1. **Selection:** Từ root, đi xuống cây theo nhánh có điểm PUCT cao nhất (xem công thức 15) cho đến khi gặp leaf node chưa được mở rộng.
2. **Expand:** Tạo tất cả node con từ các nước đi hợp lệ tại leaf, gán prior P từ policy head.
3. **Evaluate:** Gọi mạng nơ-ron một lần tại vị trí leaf để lấy value V theo công thức (8).
4. **Backpropagation:** Cập nhật N và Q ngược lên từ leaf đến root, đảo dấu tại mỗi nút vì mỗi lượt là của đối thủ:

$$N(s, a) += 1, \quad Q(s, a) = \frac{\sum \text{value}}{N(s, a)}, \quad V \leftarrow -V \text{ (khi lên nút cha)} \quad (14)$$

Công thức PUCT điều khiển việc chọn nhánh trong bước Selection, cân bằng giữa khai thác (exploit) và khám phá (explore):

$$\text{PUCT}(s, a) = Q(s, a) + c_{\text{puct}} \cdot P(s, a) \cdot \frac{\sqrt{N_{\text{parent}}}}{1 + N(s, a)} \quad (15)$$

Trong đó:

- $Q(s, a)$: giá trị trung bình từ các simulation trước (khai thác kinh nghiệm đã có).
- $P(s, a)$: prior xác suất từ policy network (định hướng ban đầu từ kiến thức đã học).
- $N(s, a)$: số lần node đã được thăm — node thăm nhiều thì ít ưu tiên hơn.
- $c_{\text{puct}} = 2,0$: hệ số cân bằng exploit-explore.

Khi $N(s, a) = 0$, phần tử khám phá rất lớn $\rightarrow$ node này được ưu tiên thăm trước. Sau khi thăm nhiều lần, Q chiếm ưu thế $\rightarrow$ khai thác node tốt nhất. Sau khi chạy đủ simulations, policy được rút ra từ phân phối visit count:

$$\pi(a \mid s) = \frac{N(s, a)}{\sum_b N(s, b)} \quad (16)$$

Phân phối này giàu thông tin hơn output thô của policy head vì nó đã được “lọc” qua hàng trăm lần thử nghiệm — đây chính là nhân học cho bước huấn luyện RL.

Dirichlet noise — khuyến khích khám phá. Nếu MCTS luôn chọn theo prior P từ policy head, mọi ván self-play sẽ đi theo cùng một đường — mạng chỉ học lại những nước nó đã biết, không bao giờ khám phá nước lạ, dữ liệu sinh ra bị lặp đi lặp lại và mạng ngừng cải thiện.

Để giải quyết, nhiễu Dirichlet được thêm vào prior của **root node** trước mỗi lượt tìm kiếm:

$$P'(a) = (1 - \varepsilon) \cdot P(a) + \varepsilon \cdot \eta(a), \quad \boldsymbol{\eta} \sim \text{Dir}(\alpha) \quad (17)$$

Trong đó:

- $\varepsilon = 0,25$: tỉ lệ nhiễu (75% prior gốc, 25% nhiễu ngẫu nhiên).
- $\alpha = 0,3$: tham số tập trung của phân phối Dirichlet. Giá trị nhỏ khiến vector nhiễu $\boldsymbol{\eta}$ phân tán đều hơn trên các nước đi, buộc MCTS thỉnh thoảng khám phá các nhánh ít được ưu tiên.
- Nhiễu **chỉ thêm vào root node**, không thêm vào các node sâu hơn. Lý do: root

là nước đi thực sự sẽ được chọn nên cần khám phá đủ rộng; các node sâu hơn chỉ là mô phỏng giả định, thêm nhiều ở đó làm giảm chất lượng đánh giá mà không mang lại lợi ích.

Kết quả: prior của root bị “xáo trộn” nhẹ mỗi lượt đi, MCTS đôi khi khám phá những nhánh mà policy head không ưu tiên, dữ liệu self-play đa dạng hơn và mạng học được nhiều chiến thuật hơn qua các iteration.

2.4.2. *Batched MCTS — tối ưu hóa phần cứng*

Triển khai MCTS thông thường gặp vấn đề nghiêm trọng: mỗi simulation cần 1 lần gọi GPU với `batch_size = 1`. Với $64 \text{ ván} \times 400 \text{ simulations} = 25.600$ lần gọi GPU đơn lẻ, GPU gần như luôn ở trạng thái idle.

Nhóm triển khai **Batched MCTS**: chạy 64 ván đồng thời, mỗi simulation step gom tất cả leaf nodes từ 64 ván thành 1 batch duy nhất, gọi GPU một lần. Ba tối ưu hóa quan trọng:

- **Fix 1 — Tránh CPU-GPU sync storm:** Xây dựng legal mask hoàn toàn trên NumPy (CPU) rồi push lên GPU một lần, thay vì gán từng phần tử trực tiếp lên GPU tensor (mỗi lần gán = 1 lần đồng bộ CPU↔GPU tốn kém).
- **Fix 2 — Tránh deep-copy board:** Dùng push/pop trực tiếp trên board gốc thay vì tạo bản sao cho mỗi simulation. Tiết kiệm hàng chục nghìn deep-copy không cần thiết trên toàn bộ quá trình self-play.
- **Fix 3 — Tái sử dụng engine:** Tạo BatchedMCTS engine một lần duy nhất cho toàn bộ vòng lặp self-play, sau mỗi nước đi chỉ reinit root thay vì tạo lại toàn bộ engine. Điều này tránh overhead khổng lồ của việc khởi tạo hàng nghìn engine objects trong suốt quá trình sinh data.

Kết quả đạt được speedup ước tính **8–12×** so với triển khai sequential.

2.4.3. *Bước 1 — Mô hình tự chơi với chính mình*

Trong mỗi iteration của vòng lặp RL, mô hình hiện tại tự chơi 64 ván cờ song song. Với mỗi nước đi, MCTS chạy 400 simulations tạo ra phân phối π , sau đó chọn nước đi theo chiến lược:

- **30 nước đầu:** Lấy mẫu ngẫu nhiên theo phân phối π (temperature = 1) để tạo ra nhiều đường khai cuộc khác nhau, làm phong phú dữ liệu huấn luyện.
- **Từ nước 30 trở đi:** Chọn greedy — nước có số lần thăm nhiều nhất — đảm bảo ván đi đến kết quả có chất lượng.

Mỗi vị trí bàn cờ được lưu cùng với phân phối MCTS và góc nhìn người đang đi. WDL label được gán sau khi ván kết thúc dựa trên kết quả thực tế.

2.4.4. Bước 2 — Huấn luyện học tăng cường

Policy Loss (RL). Trong giai đoạn RL, nhãn cho Policy Head là toàn bộ phân phối MCTS thay vì nhãn cứng một nước. Hàm mất mát là KL divergence giữa phân phối MCTS và phân phối dự đoán của mạng:

$$\mathcal{L}_{\text{policy}}^{\text{RL}} = -\frac{1}{N} \sum_s \sum_a \pi_{\text{mcts}}(a | s) \log \pi_{\text{net}}(a | s) \quad (18)$$

Value Loss vẫn dùng Cross-Entropy WDL như trong giai đoạn pretrain, nhưng nhãn lấy từ kết quả ván self-play thực tế. Tương tự giai đoạn pretrain, gradient được clip về L2-norm tối đa 1,0 sau mỗi bước để ổn định huấn luyện.

Dynamic Warmup. Khác với giai đoạn pretrain dùng warmup cố định $t_{\text{warmup}} = 1000$ bước, giai đoạn RL tính warmup động theo số bước thực tế của mỗi iteration:

$$t_{\text{warmup}}^{\text{RL}} = \max\left(1, \left\lfloor \frac{|\text{loader}| \times E}{10} \right\rfloor\right) \quad (19)$$

Trong đó $|\text{loader}|$ là số batch trong DataLoader thực tế và $E = 3$ là số epoch. Cách tính này tự động scale theo kích thước replay buffer: khi buffer còn ít data (iteration đầu), warmup ngắn; khi buffer đầy (4 iteration), warmup dài hơn tương ứng. Tránh được tình trạng warmup quá dài so với tổng số bước thực tế — vốn gây ra learning rate luôn ở vùng tăng tuyến tính mà không bao giờ đạt đỉnh.

Replay Buffer với trọng số. Để tránh catastrophic forgetting, nhóm dùng sliding window replay buffer giữ data của tối đa 4 iteration gần nhất (`maxlen=4`). Data mới được ưu tiên hơn data cũ thông qua `WeightedRandomSampler` với trọng số tuyến tính từ 0,25 (cũ nhất) đến 1,0 (mới nhất). Sampler được cấu hình với `replacement=True` — cần thiết để các position có trọng số cao có thể xuất hiện nhiều hơn một lần trong cùng một epoch, đảm bảo phân phối sample thực sự phản ánh trọng số đã gán thay vì chỉ xáo thứ tự.

2.4.5. Bước 3 — So sánh và cập nhật best model

Sau khi huấn luyện xong, model mới đấu với best model hiện tại trong 40 ván (20 ván cầm Trắng, 20 ván cầm Đen) với 200 simulations mỗi nước. Để tránh hiện tượng các ván đấu bị lặp do tính tất định của mô hình (cùng vị trí luôn chọn cùng một nước),

mỗi ván được khởi đầu từ một khai cuộc ngẫu nhiên khác nhau lấy từ thư viện 20 đường khai cuộc phổ biến. Ngoài ra, tham số `add_noise=True` được bật trong MCTS để đưa thêm nhiễu Dirichlet vào root node, phá vỡ hoàn toàn tính tất định trong quá trình tìm kiếm. Kết hợp hai biện pháp này đảm bảo 40 ván eval phản ánh thực lực tương đối của hai model thay vì chỉ là bản sao của nhau.

Win rate được tính:

$$\text{Win Rate} = \frac{W + 0,5 \cdot D}{N} \quad (20)$$

Trong đó W là số ván thắng, D là số ván hòa, N là tổng số ván. Ngưỡng thay thế đặt là **55%** thay vì 50% vì với 40 ván thì variance vẫn còn, cần thắng rõ ràng mới đủ độ tin cậy thống kê. Nếu $\text{Win Rate} \geq 0,55$, model mới trở thành best model cho iteration tiếp theo.

2.5. Các cơ chế tối ưu tốc độ và phần cứng

Toàn bộ quá trình huấn luyện được thực hiện trên môi trường Kaggle và Google Colab với GPU T4. Để tối đa hóa hiệu quả sử dụng phần cứng, nhóm áp dụng đồng thời Mixed Precision (AMP), Batched MCTS, và Chunk DataLoader như đã trình bày ở các mục trên. Kết hợp lại, các kỹ thuật này giúp pipeline hoàn chỉnh có thể chạy trong giới hạn tài nguyên của môi trường free-tier.

Tóm tắt vòng lặp RL (lặp lại 10 lần):

1. **Sinh data:** 64 ván self-play với Batched MCTS (400 sims/nước) $\rightarrow \sim 15.000$ vị trí/iteration.
2. **Huấn luyện:** Copy best model $\rightarrow$ 3 epoch AdamW với dynamic warmup trên replay buffer có trọng số.
3. **Đánh giá:** 40 ván new model vs. best model (khai cuộc ngẫu nhiên, `add_noise=True`) $\rightarrow$ tính Win Rate.
4. **Cập nhật:** Nếu $\text{Win Rate} \geq 0,55$ thì thay thế best model, ngược lại giữ nguyên.